

EVERYDAY TOOLS — TOOL BUILD (MECHANICS)

Tate Break

Mechanics prototype, back-end architecture notes, and submission bundle

Tool: Tate Break

Module: Everyday Tools — Summit on Human Dignity

Assignment: Tool Build (mechanics)

Author: Tate Riddell

Live deployed link: <https://claude.ai/artifact/JBJwbkipTkLsmoyFoY6Ndo>

GitHub repository: [tate-break-mechanics](#) (see Submission Instructions, p. 6, for the one remaining manual step)

Screen recording: [tate-break-demo.mp4](#), 45 seconds — attached separately alongside this PDF

WHAT WAS BUILT

A standalone, dependency-free web prototype of Tate Break's actual interruption mechanic — the same trigger-and-companion loop already shipped as a working Chrome extension, rebuilt here with input, logic, and output pulled apart into their own files, per this assignment's architecture requirement.

Core loop: scroll a simulated feed. Tate appears once shortly after the page loads, and again every fixed 10 scrolls. He walks on, lies down, covers the feed, leads a 15-second breathing pause with cycling cues, then gets up and leaves — the feed resumes exactly where it was.

One clearly implemented core function: the fixed scroll-count trigger → on-screen companion break → automatic resume cycle.

Defined input: mouse-wheel / trackpad scroll events on the page. **Defined processing:** a three-state engine counting scrolls toward a disclosed fixed target. **Defined output:** Tate's walk-on/ breathing/walk-off sequence plus an always-visible tally.

Visible choice point: a small pill fixed to the top-right corner ("Tate in N scrolls") shows the exact, real count at all times — never hidden, never randomized — so the person always knows how close the next interruption is before it happens.

ARCHITECTURE — INPUT / LOGIC / OUTPUT

```
src/input/scrollInput.js    INPUT LAYER
src/logic/breakEngine.js    LOGIC LAYER
src/output/tateRenderer.js  OUTPUT LAYER
main.js                    wiring only — no behavior of its own
```

Input layer — scrollInput.js

Listens for raw `wheel` events and turns a noisy burst of native events (one real scroll gesture can fire dozens in a few milliseconds) into a single clean "a scroll happened" signal, using a 350ms gap to tell one gesture from the next. Knows nothing about Tate, counting, or timers.

Logic layer — breakEngine.js

The actual state machine: a scroll counter, a fixed target (10), an open-delay timer, and a break-duration timer. Two states — `IDLE` (counting) and `BREAK_ACTIVE` (a break is running; new scroll signals are ignored, not queued, until it ends). No DOM references anywhere in this file — the same fixed-count design already tested and shipped in the real extension's `content.js`, lifted out of the browser entirely.

Output layer — tateRenderer.js

Everything about how it looks: the walk-on/lie-down/breathing/walk-off sequence, the cover image, and the tally pill. Receives events from the logic layer and never decides when those events happen.

Wiring — main.js

Creates one of each layer and connects them with plain callbacks. Also builds the cosmetic simulated feed, which the mechanic doesn't actually depend on — swap in a real feed and nothing else in the project needs to change.

BACK-END THINKING FIRST

What data does this tool actually need to function?

One integer (scrolls since the last break) and one state flag (is a break active). No user profile, no history of past sessions, no record of what was being scrolled.

Where does that data need to live?

Nowhere outside the browser tab's own memory — plain JavaScript variables closed over inside `createBreakEngine()`. Never written to a file, a cookie, localStorage, or a server.

Does it need to be temporary or persistent?

Temporary, on purpose. Reloading the page resets the count to zero and returns to `IDLE` — matching the shipped extension's own documented behavior.

Does it need to remember anything across sessions?

No, by design. Remembering break history or "how many times you needed this today" would turn a companion into a tracker, which the tool's own Dignity Clause rules out.

Does this tool require any AI or inference?

No. Tate Break is a scripted, deterministic state machine — built explicitly as a non-AI stand-in for a separate AI-companion project. Nothing in this prototype calls a model.

How many API calls does this tool expect to make, and when?

Zero. There is no network layer anywhere in `src/` — no fetch, no XHR, no analytics beacon.

What happens if an API call fails?

Not applicable today, since none exist — but the governing principle for any future one is **fail open, never fail closed**: a break tool that locked the feed on an error would violate its own Refusal Clause ("never locks the feed permanently"). A future network dependency would default to the last-known-good local config and let scrolling continue rather than block on a failed request.

BEHAVIOR INTEGRITY CHECK

Cross-referenced against Tate Break's own `SYSTEM_CHARTER.md` (written for the prior Interface Ritual Sketch submission, since Tate Break branched off mid-module before a formal Tool Intent Statement existed for it):

Charter says	This prototype does
"Fires once shortly after the page opens, and then again every 10 scrolls... a fixed, disclosed number, not a hidden one."	<code>openDelayMs: 1500</code> and <code>scrollTarget: 10</code> in <code>main.js</code> , both plain constants, both shown via the tally pill.
"No... scroll history saved beyond the live count for the current break cycle (which resets to zero the moment he appears)."	<code>endBreak()</code> sets <code>scrollCount = 0</code> the moment a break ends; nothing is written anywhere before that reset.
"Never locks the feed permanently... never nags outside its one 15-second window."	<code>BREAK_ACTIVE</code> always returns to <code>IDLE</code> on a fixed timer — no code path leaves it active indefinitely.
"The trigger count is fixed at 10... it isn't hidden or randomized."	The tally pill renders the real <code>scrollCount</code> / <code>scrollTarget</code> values on every scroll.

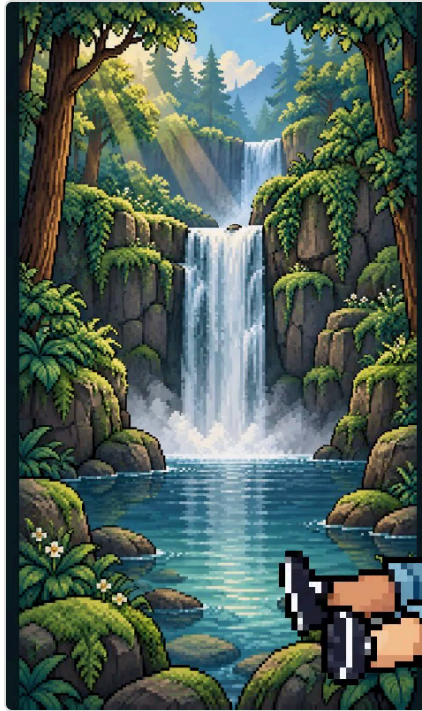
No gaps found between the stated charter and the built mechanic for the one function this prototype implements. The charter's Notes/ Reminders/Goals hub is a separate feature of the full Chrome extension and is intentionally out of scope here.

SCREEN RECORDING — STORYBOARD

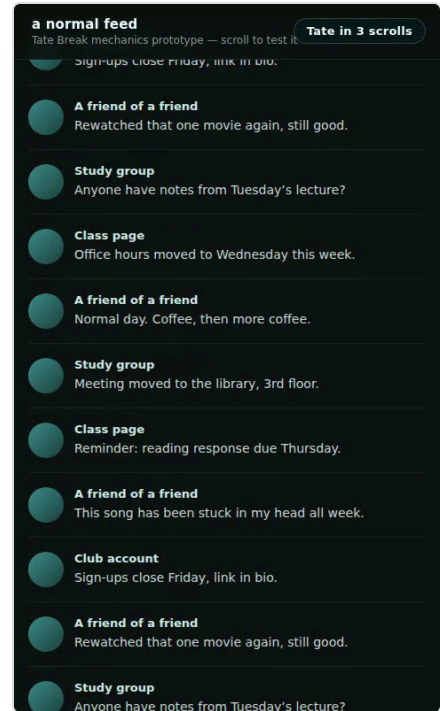
Full recording: `tate-break-demo.mp4` , 45 seconds (under the 2-minute limit), attached alongside this PDF — a PDF page can't play video, so the frames below are a storyboard of it, in order.



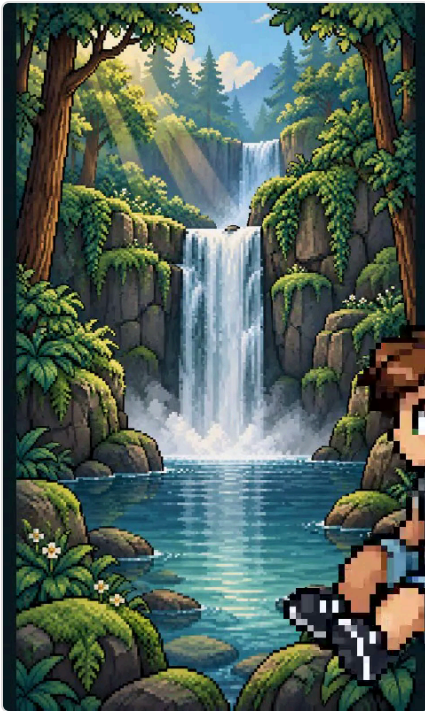
0:02 — opens the page; Tate walks on 1.5s later, unprompted



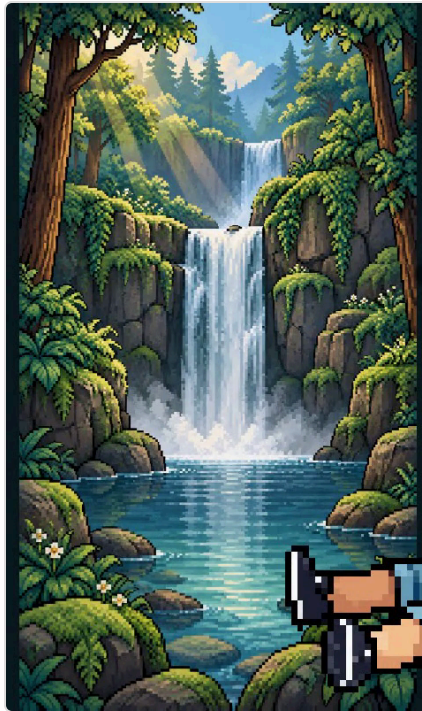
0:04 — lies down, breathing cues begin



0:23 — scrolling the feed; tally reads "Tate in 3 scrolls"



0:27 — the 10th scroll fires the second break



0:30 — second break in progress, same sequence

SUBMISSION INSTRUCTIONS

Everything above was built with what's actually available in this environment. One step needs to happen on your own machine, honestly flagged rather than faked:

1. **Push the repo to GitHub.** A local git repository already exists with 7 clean, sequential commits (input layer → logic layer → output layer → wiring → assets → build scripts → README) in the `tate-break-mechanics` folder on your Desktop. This environment has no GitHub login and can't create one on your behalf, so from a terminal in that folder: create a new empty repository on github.com (no README/license, so it doesn't conflict), then run `git remote add origin <your-new-repo-url>` and `git push -u origin master`. The commit history will carry over exactly as built.
2. **Live link:** already deployed — `https://claude.ai/artifact/JBJwbkipTkLsmoyFoY6Ndo`. No action needed unless your instructor requires a GitHub Pages URL specifically, in which case GitHub's own Pages setting (Settings → Pages → deploy from `master`) will serve `index.html` directly once the repo is pushed.
3. **Submit this PDF** plus `tate-break-demo.mp4` as two attachments (or one zip, if your LMS prefers a single upload) to the assignment.

Why this one step is manual: this environment is not signed into any GitHub account, and creating an account or entering credentials on your behalf isn't something it does — the repo, its commit history, and everything else are done; only the login-gated push is left to you.